3.1 Abstract Computation Model

We consider a concurrent system composed of a finite set of *processes* (or threads). Each process executes a deterministic program, whereby statements act on the *state* of the system, which is made up of the local states of each process and the shared state of the system (*state* will refer to the global state, unless explicitly specified otherwise). We do not restrict the system to a specific mode of process interaction, allowing instead the use of shared variables, messages, etc. We assume that the state space does not contain cycles, and that executions have bounded length. This means that executions must terminate by themselves within a bounded number of steps. Our algorithms, as presented in this article, are unsound if executions are simply truncated beyond some given bound. Soundness, in the sense of covering all executions up to a given length, can be restored at the expense of refining the control over execution scheduling and exploring some executions beyond the imposed bound; the details of this are beyond the scope of this article.

Let Σ be the set of states of the system. The system has a unique *initial state* $s_0 \in \Sigma$. We assume that the program executed by a process p can be represented as a partial function $execute_p : \Sigma \mapsto \Sigma$ which moves the system from one state to a subsequent state. Each such application of the function $execute_p$ represents an atomic *execution step* of process p, which may depend on and affect the state. We let each execution step (or just *step* for short) represent the combined effect of some global statement together with the following finite sequence of local statements (that only access and affect the local state of the process), ending just before the next global statement. This avoids consideration of interleavings of local statements of different processes in the analysis. Such an optimization is common in tools (e.g., Verisoft [Godefroid 1997]).

An *execution sequence* E of a system is a finite sequence of execution steps of its processes that is performed from the initial state s_0. Since each execution step is deterministic, an execution sequence E is uniquely characterized by the sequence of processes that perform steps in E. For instance, $p.p.q$ denotes the execution sequence where first p performs two steps, followed by a step of q. The sequence of processes that perform steps in E also uniquely determine the state of the system after E, which is denoted $s_{[E]}$. The execution of a process is said to *block* in some state s if the process cannot continue (i.e., $execute_p(s)$ is undefined): for example, trying to receive a message in a state where the message queue is empty. For a state s, let $enabled(s)$ denote the set of processes p that are enabled in s (i.e., for which $execute_p(s)$ is defined). We say that E is *maximal* if $enabled(s_{[E]}) = \emptyset$, i.e., no process is enabled after E. We use . to denote concatenation of sequences of processes. Thus, if p is not blocked after E, then $E.p$ is an execution sequence. We use $w, w', \ldots$ to range over arbitrary sequences of processes.

An *event* of E is a particular occurrence of a process in E. More precisely, an event is a pair $\langle p, i \rangle$, representing the ith occurrence of process p in the execution sequence. We use $e, e', \ldots$ to range over events, as well as:

- $E \vdash w$ to denote that $E.w$ is an execution sequence.
- $w \setminus p$ to denote the sequence w with its first occurrence of p removed.
- $dom(E)$ to denote the set of events $\langle p, i \rangle$ which are in E, i.e., $\langle p, i \rangle \in dom(E)$ iff E contains at least i steps of p.
- $dom_{[E]}(w)$, where $E.w$ is an execution sequence, to denote $dom(E.w) \setminus dom(E)$, i.e., the events in $E.w$ which are in w.
- $next_{[E]}(p)$ to denote $dom_{[E]}(p)$ as a special case.
- $\widehat{e}$ to denote the process p of an event $e = \langle p, i \rangle$.
- $op_{[E]}(e)$ to denote the operation performed by event e in execution sequence E.
- $<_E$ to denote the total order between events in E, i.e., $e <_E e'$ denotes that e occurs before e' in E.